

PRÜFUNG SPRG HSLU

Secure Requirement Engineering & Secure Programming

Datum: xx.xx.xxxx

Zeit: 90 Minuten

Dozent: Silvan Leuenberger / Michael Lehmann

Vorname: _____

Nachname: _____

Information, Ablauf, Bewertung

Bitte lesen Sie die Fragen genau durch. Bei den Multiple Choice Aufgaben sind jeweils eine oder mehrere Antworten zutreffend, eine falsche bzw. zu wenig angewählte Antwort gibt einen anteilmässigen Abzug. Die anderen Aufgaben sind frei zu beantworten mit Angabe der Nummer (z. Bsp. «1») und der entsprechenden Lösung auf dem leeren Lösungsblatt. Bei Multiple Choice Aufgaben einfach die Nennung der richtige/n Antwort/en (z. Bsp. a ,d) auf dem Lösungsblatt bei der entsprechenden Aufgabennummer.

Insgesamt werden 90 Punkte vergeben (45 pro Teil), jeweils pro Frage mit (x) angegeben. 60 Punkte von den Total 90 Punkten müssen erreicht werden, um die Prüfung zu bestehen.

Abgabe: Prüfungsblatt zuerst und dann das Lösungsblatt in korrekter Reihenfolge wieder zusammenstellen und mit der Büroklammer fixieren. In den letzten 5' bitte sitzen bleiben.

Kleiner Tipp: 1 Punkt entspricht ca. 1 Minute Zeit für das Zeitmanagement

 **Viel Erfolg!** 

Teil 1: Secure Requirement Engineering

Lesen Sie die Anweisungen sorgfältig: Bevor Sie mit der Beantwortung der Fragen beginnen, nehmen Sie sich die Zeit, die Anweisungen und jede Frage gründlich zu lesen, um Missverständnisse zu vermeiden.

Zeitmanagement: Teilen Sie Ihre Zeit gut ein und achten Sie darauf, dass Sie für jede Aufgabe genügend Zeit haben. Planen Sie auch Pufferzeiten für Überprüfungen ein.

Bleiben Sie ruhig und fokussiert: Lassen Sie sich nicht von Stress oder Nervosität überwältigen. Atmen Sie tief durch und konzentrieren Sie sich auf eine Frage nach der anderen.

Überprüfen Sie Ihre Antworten: Nutzen Sie die letzten Minuten der Prüfung, um Ihre Antworten noch einmal durchzugehen und eventuelle Fehler zu korrigieren.

In der Kürze liegt die Würze. Kurz und prägnant wird besser bewertet als Ausschweifend und unklar.

- 1) Warum ist es sinnvoll Requirement Engineering speziell im Bereich IT-Security durchzuführen, was ist der Nutzen davon? (1)
- 2) Beschreiben Sie Ihr vorgehen beim Threatmodelling (5)
- 3) Nenne zu den **2 genannten** Techniken, um Requirements zu finden, allfälliger Vor- und Nachteile (4)
- 4) Wir haben das DREAD Vorgehen kennen gelernt und angewandt. (2)
 - a. Was ist der Nutzen von diesem Vorgehen?
 - b. Wann und Warum ist es hilfreich?
- 5) Wir haben die Begriffe Vulnerability, Threat und die dazugehörige Action besprochen. Machen Sie dazu **5** Beispiel für eine **Vulnerability**, den daraus resultierenden **Threats**, und die dazu gehörige «**Threat – Action**» (10)
- 6) Beschreibe zwei Beispiel Usecases im Bereich IT-Sicherheit (6):
 - Wie wir dies auch im Unterricht gemacht haben, ohne genau die Beispiele zu nennen, welche wir in den Folien genutzt haben (3)?
 - Beschreibe dazu den Misusecase (3)
- 7) Wir haben die **7** Schritte des SDLC nach Microsoft kennengelernt und den Ansatz, der dadurch verfolgt wird. Beschreibe nachfolgendes in eigenen Worten (3)

8) Übung zum Threat Modelling: Wartenautomat (14)

Szenario: Wir beschäftigen uns mit einem Wartenautomat, (im Sinne und Art wie die bekannten Selecta Automaten) an einem öffentlichen Bahnhof. Der Automat akzeptiert verschiedene Zahlungsmethoden wie Bargeld, Kreditkarten und mobile Zahlungsdienste. Er ist mit dem zentralen Netzwerk des Herstellers verbunden, um Daten über Preise und Artikel zu aktualisieren sowie Transaktionen zu verarbeiten und sich für Wartungsaufgaben per Remote Verbindung zu melden.

Ziel der Übung: Identifizieren und analysieren Sie potenzielle Bedrohungen für den Automaten, bewerten Sie die Risiken und entwickeln Sie Massnahmen zur Risikominderung.

Schritte der Übung

1. Systembeschreibung und Identifizierung von Assets

- Beschreiben Sie die Komponenten des Automaten. Identifizieren Sie die wichtigsten Assets (z.B. Zahlungsdaten, Kundeninformationen, Betriebssoftware).

2. Erstellung eines Datenflussdiagramms (DFD)

- Zeichnen Sie ein DFD, das die Interaktionen zwischen dem Automaten, den Nutzern, den Zahlungsanbietern und dem zentralen Netzwerk des Anbieter zeigt. Markieren Sie die Datenflüsse, Speicherung und Verarbeitungsstellen.

3. Identifizierung von Bedrohungen mit STRIDE

- Verwenden Sie das STRIDE-Modell (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege), um mögliche Bedrohungen für den Automaten zu identifizieren.

4. Bewertung der Risiken mit DREAD

- Bewerten Sie die identifizierten Bedrohungen hinsichtlich ihrer Wahrscheinlichkeit und der möglichen Auswirkungen.

5. Entwicklung von Gegenmassnahmen

- Entwickeln Sie Massnahmen zur Minderung der identifizierten Bedrohungen.

Teil 2: Secure Programing

1. Welche(r) OAuth-Flow(s) ermöglicht/en die Nutzung durch ein Gerät ohne direkten Benutzerzugriff? (2)
 - a. Authorisation Code Flow
 - b. Implicit Flow
 - c. Client Credential Flow
 - d. Device Code Flow
 - e. Keine
2. Welche Aussage(n) über Cross-Site Request Forgery (CSRF) ist/sind korrekt? (2)
 - a. CSRF kann durch SameSite-Cookies und Input Validation verhindert werden.
 - b. CSRF erfordert immer aktive Benutzerinteraktion.
 - c. Ein CSRF-Token ist eine effektive Gegenmaßnahme.
 - d. CSRF betrifft nur POST-Anfragen.
 - e. Keine der Aussagen ist korrekt.
3. Was ist ein Hauptmerkmal einer Zero Trust-Sicherheitsarchitektur? (2)
 - a. Ständige Authentifizierungsüberprüfung
 - b. Konzentration auf Netzwerkperimeter
 - c. Zugang auf Basis von Vertrauenszonen
 - d. Anwendung von Rollen und Berechtigungen
 - e. Keine dieser Merkmale trifft zu.
4. Identifizieren Sie die Sicherheitslücke im folgenden Code und beschreiben Sie, wie diese behoben werden könnte (kein Code, nur Mitigation). (3)

```
def process_input(user_input):  
    command = f"echo {user_input}"  
    return os.popen(command).read()  
  
user_input = input("Enter some text: ")  
result = process_input(user_input)  
print(result)
```

5. Welche(r) Mechanismus/Maßnahmen helfen gegen SQL-Injection? (2)
 - a. Prepared Statements
 - b. Input Validation
 - c. Stored Procedures
 - d. Einsatz von CAPTCHAs
6. Eine Webapplikation enthält folgenden Code, um Benutzereingaben für Dateioperationen zu verarbeiten:

```
def read_file(filename):  
    with open(filename, 'r') as file:  
        return file.read()  
  
user_input = input("Enter the file name: ")  
print(read_file(user_input))
```

Was für eine Schwachstelle ist im Code Beispiel vorhanden und wie könnte diese verhindert werden (kein Code, nur Beschreibung der Mitigation)? (4)

7. Welche der folgenden Aussagen trifft zu? (2)

- a. ABAC ermöglicht flexiblere Zugriffskontrolle als RBAC.
 - b. Bei RBAC werden Rechte Gruppen zugeordnet.
 - c. ABAC ist immer sicherer als RBAC.
 - d. Keine der Aussagen trifft zu.
8. In einer Webapplikation gibt es ein Eingabefeld, das SQL-Abfragen verarbeitet. Der folgende Code zeigt die Implementierung der Datenabfrage:

```
import sqlite3

def get_user_data(user_id):
    connection = sqlite3.connect('database.db')
    cursor = connection.cursor()
    query = f"SELECT * FROM user_sessions WHERE user_id = {user_id}"
    cursor.execute(query)
    return cursor.fetchall()

user_input = input("Enter your user ID: ")
print(get_user_data(user_input))
```

Welche der folgenden Eingaben würden eine SQL-Injection ausnutzen, um alle Daten aus der Tabelle user_sessions auszulesen? (2)

- a. 1 OR 1=1
 - b. 1; DROP TABLE user_sessions
 - c. '; SELECT * FROM user_sessions –
 - d. Keine der genannten Eingaben
9. Beim Umgang mit JSON Web Tokens (JWT) sind welche Aspekte entscheidend? (2)
- a. Signaturintegrität
 - b. Ablaufdatum des Tokens
 - c. Der salted Hash des Passworts
 - d. Validierung des Token-Issuers
10. Unter welchen Bedingungen ist eine Cross-Site Scripting (XSS)-Schwachstelle möglich? (2)
- a. HTML-Attribute enthalten Benutzerinput ohne Validierung.
 - b. Kein CORS wird angewendet.
 - c. Browser-Sicherheitsfunktionen deaktiviert.
 - d. Verwendung von dynamisch generierten Script-Tags.
11. Welche Aussage(n) zu Dependency Management ist/sind korrekt? (2)
- a. Alle Abhängigkeiten sollten automatisch aktualisiert und direkt deployed werden.
 - b. Es ist ausreichend, bei jedem Build die Schwachstellen von Libraries zu scannen.
 - c. Interne Repositories verbessern die Sicherheit nicht.
 - d. Keine der Aussagen ist korrekt.
12. Was ist der Hauptunterschied zwischen Path Traversal und Local File Inclusion (LFI)? (2)
- a. LFI erlaubt das Einfügen von externen Dateien, Path Traversal jedoch nicht.
 - b. Path Traversal betrifft nur Betriebssystemdateien, LFI betrifft nur Anwendungsdateien.
 - c. Beide betreffen nur lokale Dateien und sind gleichwertig.
 - d. Path Traversal ermöglicht das Lesen beliebiger Dateien, während LFI Dateien innerhalb des Webroot lädt.

13. Was sind gute Merkmale einer Passwort-Policy? (2)
- Passwörter müssen Sonderzeichen enthalten.
 - Passwörter sollten länger als 12 Zeichen sein.
 - Keine dieser Elemente.
14. Folgender Java Code implementiert eine Suche nach einem Bankkonto in einer Backend Komponente einer Bank Applikation. Der Output ist die Liste aller Konten des eingeloggten Benutzers und deren Kontostände mit einem gewissen Namen.

```
String currentUser = principal.getName();
String accountName = request.getParameter("accountName");
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery(
    "SELECT ACCOUNT_ID, BALANCE, NAME, USERID FROM ACCOUNT " +
    "WHERE NAME LIKE '%" + accountName + "%' " +
    "AND USERID='" + currentUser + "'"
);
```

Die Variable *currentUser* wird von der Security-Komponente des Services gesetzt (Wert ist der eingeloggte Benutzer) und kann in dieser Aufgabe nicht angepasst werden. Der Wert *accountName* für die Suche nach dem Konto wird vom User in einem Eingabefeld im Browser erfasst.

Nehmen wir an, der Service ist anfällig auf gewisse SQL Injections. Welchen Wert wählen Sie für das Feld *accountName*, damit die Konten (Tabelle ACCOUNT) und deren Kontostand des Benutzers mit *USERID=user1234* und nur diese angezeigt werden? (6)

- ' OR '1'='1' –
 - ' UNION SELECT ACCOUNT_ID, BALANCE FROM ACCOUNT WHERE USERID='user1234' --
 - '%user1234%'
 - ' OR USERID='user1234' –
15. Identifizieren Sie die Sicherheitslücke im folgenden Code und beschreiben Sie, wie diese behoben werden könnte (kein Code, nur Mitigation). Geben Sie auch an, wie die Schwachstelle konkret ausgenutzt werden könnte. (6)

```
function displaySearchResults() {
    const searchQuery = new URLSearchParams(window.location.search)
        .get('query');
    document.getElementById('results').innerHTML =
        "Ergebnisse für: " + searchQuery;
}

displaySearchResults();
```

16. Mass Assignment ist ein häufiger Fehler in Webapplikationen. Welche Aussage(n) dazu ist/sind korrekt? (2)
- Mass Assignment erlaubt einem Angreifer, sensible Felder wie *isAdmin* zu überschreiben.
 - Der Fehler tritt nur bei SQL-Datenbanken auf.
 - White- und Blacklisting können Mass Assignment verhindern.

- d. Es betrifft nur Anwendungen ohne Benutzer-Authentifizierung.
17. Buffer Overflow ist eine häufige Schwachstelle in unsicherer Software. Welche Aussage(n) dazu ist/sind korrekt? (2)
- a. Ein Buffer Overflow tritt auf, wenn mehr Daten geschrieben werden, als ein Speicherbereich aufnehmen kann.
 - b. Schutzmechanismen wie Stack Canaries und ASLR können Buffer Overflows verhindern.
 - c. Buffer Overflows betreffen nur Anwendungen in Hochsprachen wie Python.
 - d. DEP (Data Execution Prevention) schützt vor der Ausführung von schädlichem Code